



People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Mohamed Seddik Ben Yahia University - Jijel

Faculty of Sciences and Technology
Department of Computer Science

School Management System

UML Analysis and Design Report

Specialty: Computer Science (Informatics)

Level: 3rd Year License

Academic Session: 2025 - 2026

Prepared by

1. Bouremouz Mouhammed
2. Debieche Amine
3. Bessibes Mouin
4. Bouzekria Sohib
5. Side Ahmed Cheabani
6. Chemsse Adden Marriche

Under the supervision of

Dr. El Hillali Kerkouche

Table of Contents

Functional Models: Use Case Diagrams	3
Admin Use Case Diagram	4
Receptionist Use Case Diagram	5
Accountant Use Case Diagram	6
Teacher Use Case Diagram	7
Student Use Case Diagram	8
Parent Use Case Diagram	9
Structural Model: Class Diagram	11
School Management System Class Diagram	12
Dynamic Models: Sequence Diagrams	14
Admin Sequence Diagram	15
Receptionist Sequence Diagram	16
Accountant Sequence Diagram	17
Teacher Sequence Diagram	18
Student Sequence Diagram	19
Parent Sequence Diagram	20
User Interface: Navigation Diagram	22
Role-Based Navigation Diagram	23
Technology Stack and Project Repository	25
Conclusion	26

Introduction

This report presents the UML analysis of the School Management System. The project is organized around six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent. The diagrams document the functional responsibilities of each role, the backend structure, the runtime communication between frontend and backend, and the navigation flow of the application.

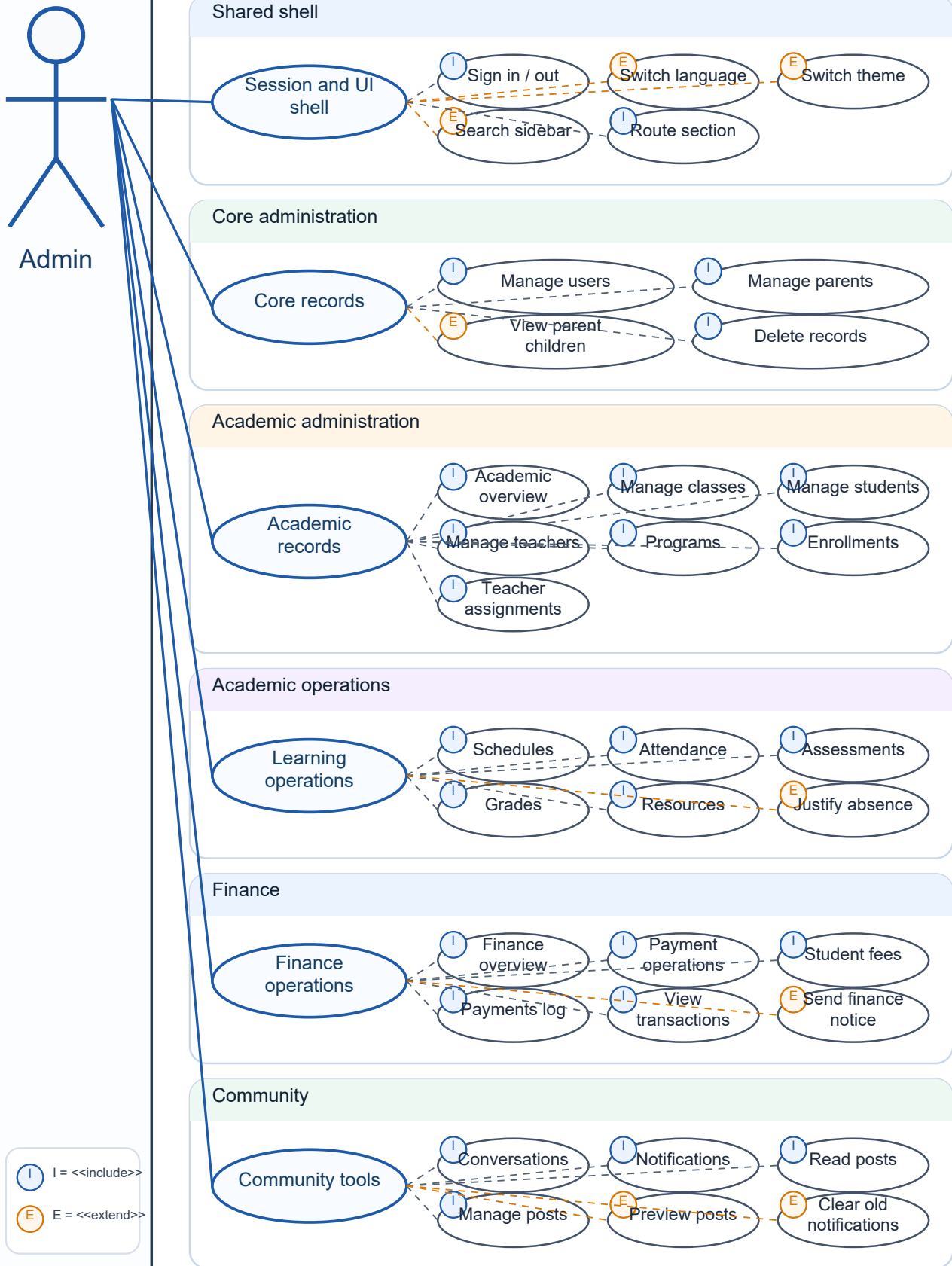
The report is structured into use case diagrams, a class diagram, sequence diagrams, the navigation diagram, a technology stack summary, and a concise conclusion.

Functional Models: Use Case Diagrams

The use case diagrams define the visible services of the School Management System for each role. They show the functional boundary of every dashboard and clarify which operations belong to administration, reception, accounting, teaching, learner self-service, and parent follow-up.

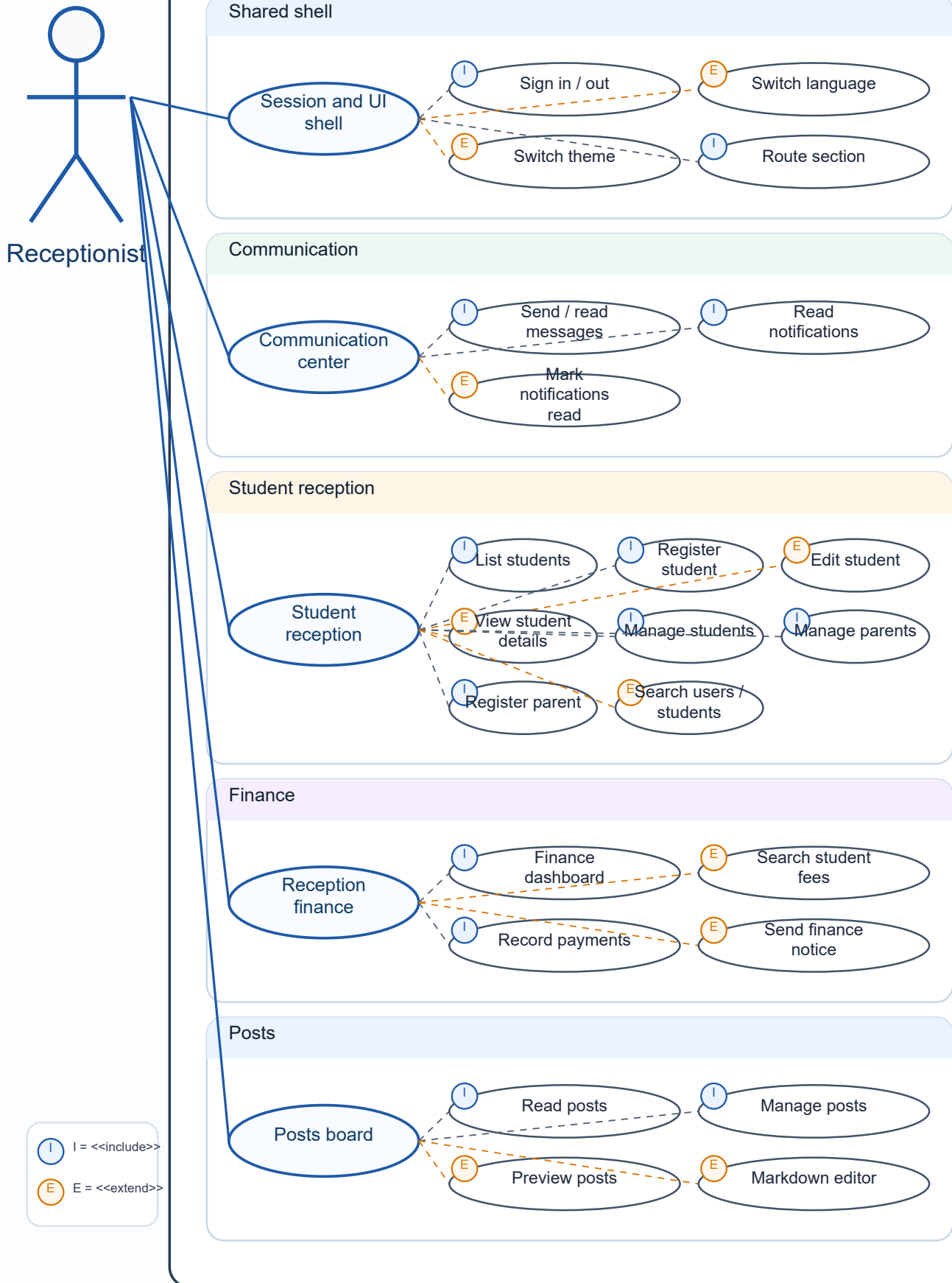
Use Case Diagram - Admin

School Management System boundary



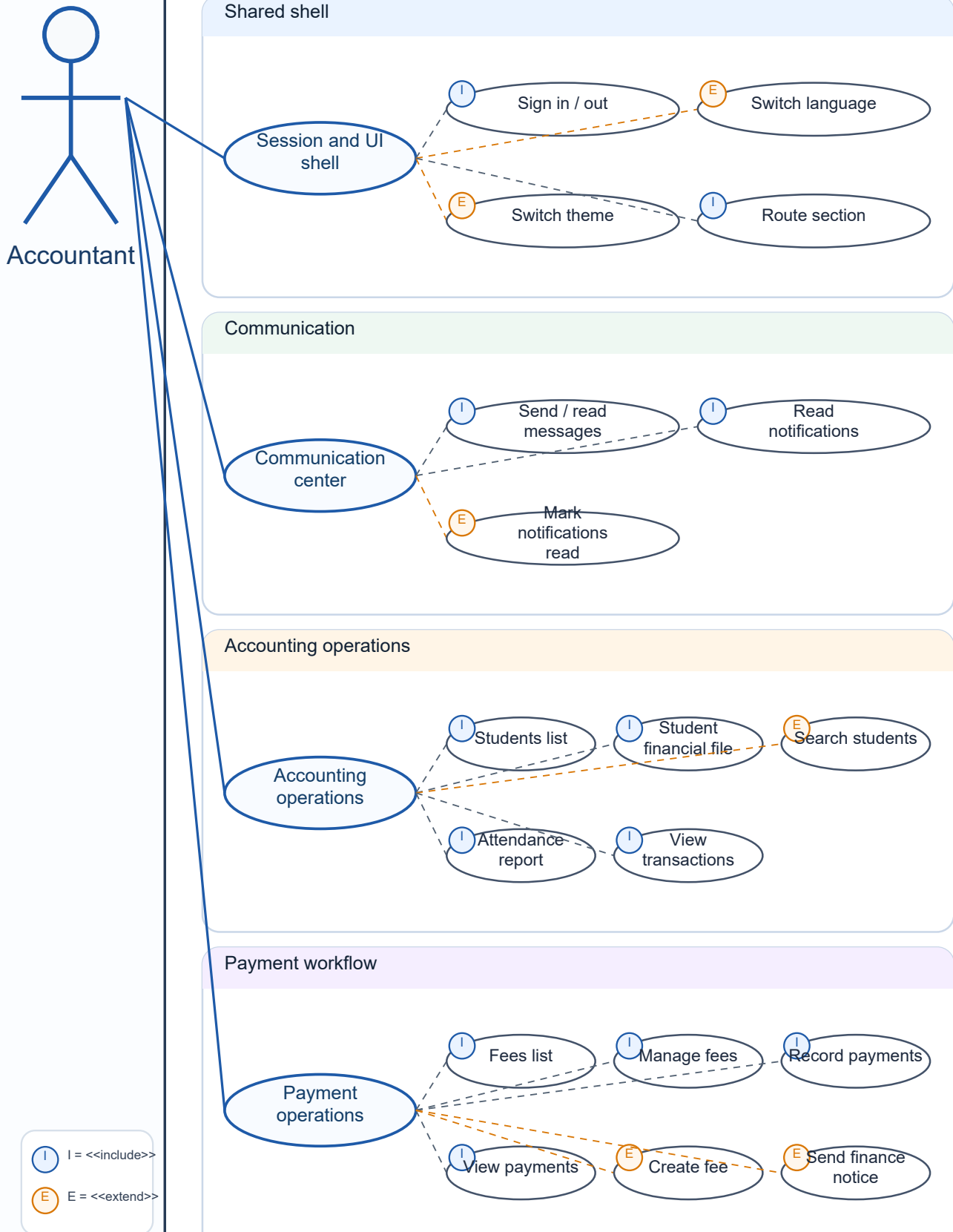
Use Case Diagram - Receptionist

School Management System boundary



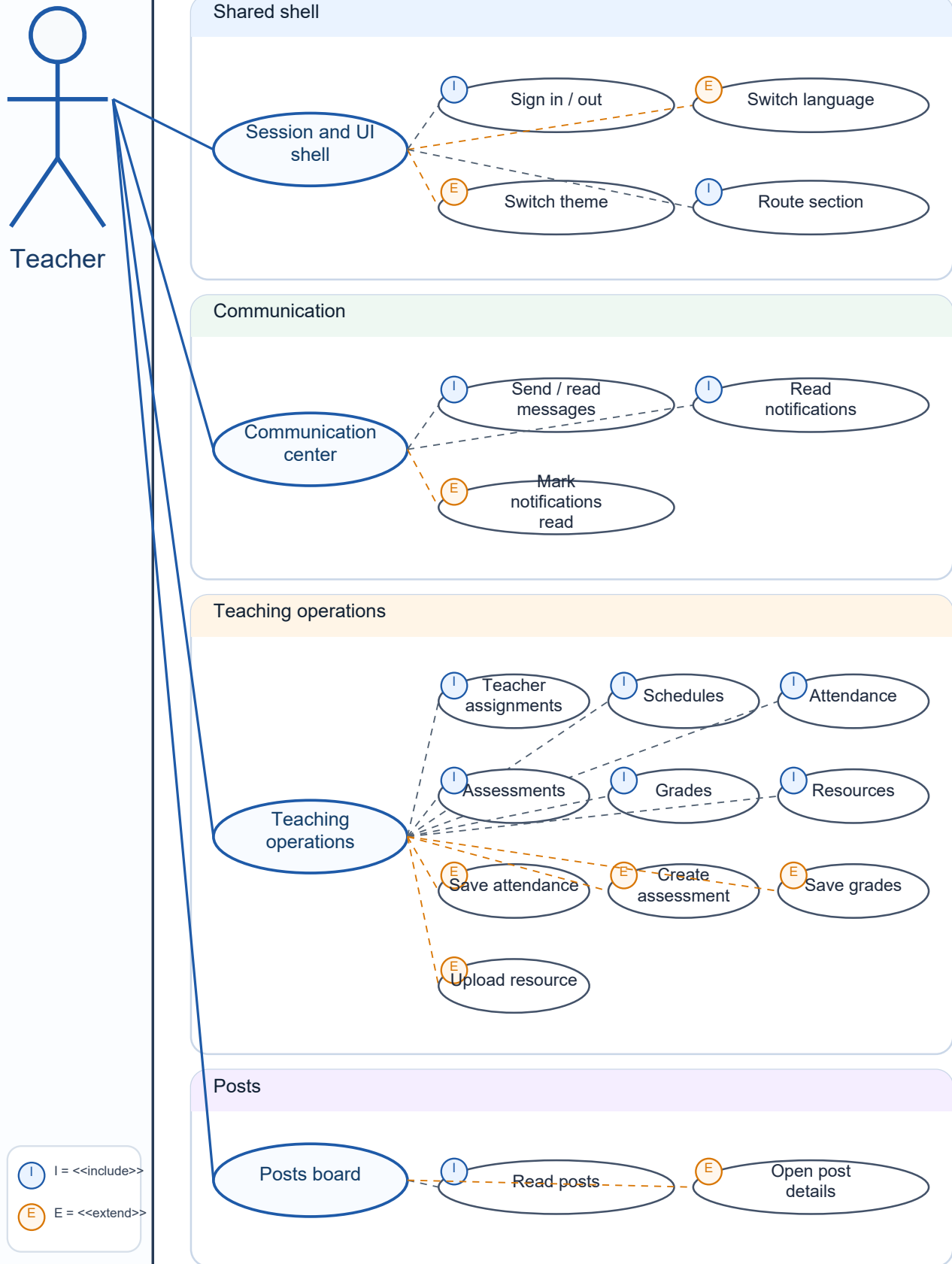
Use Case Diagram - Accountant

School Management System boundary



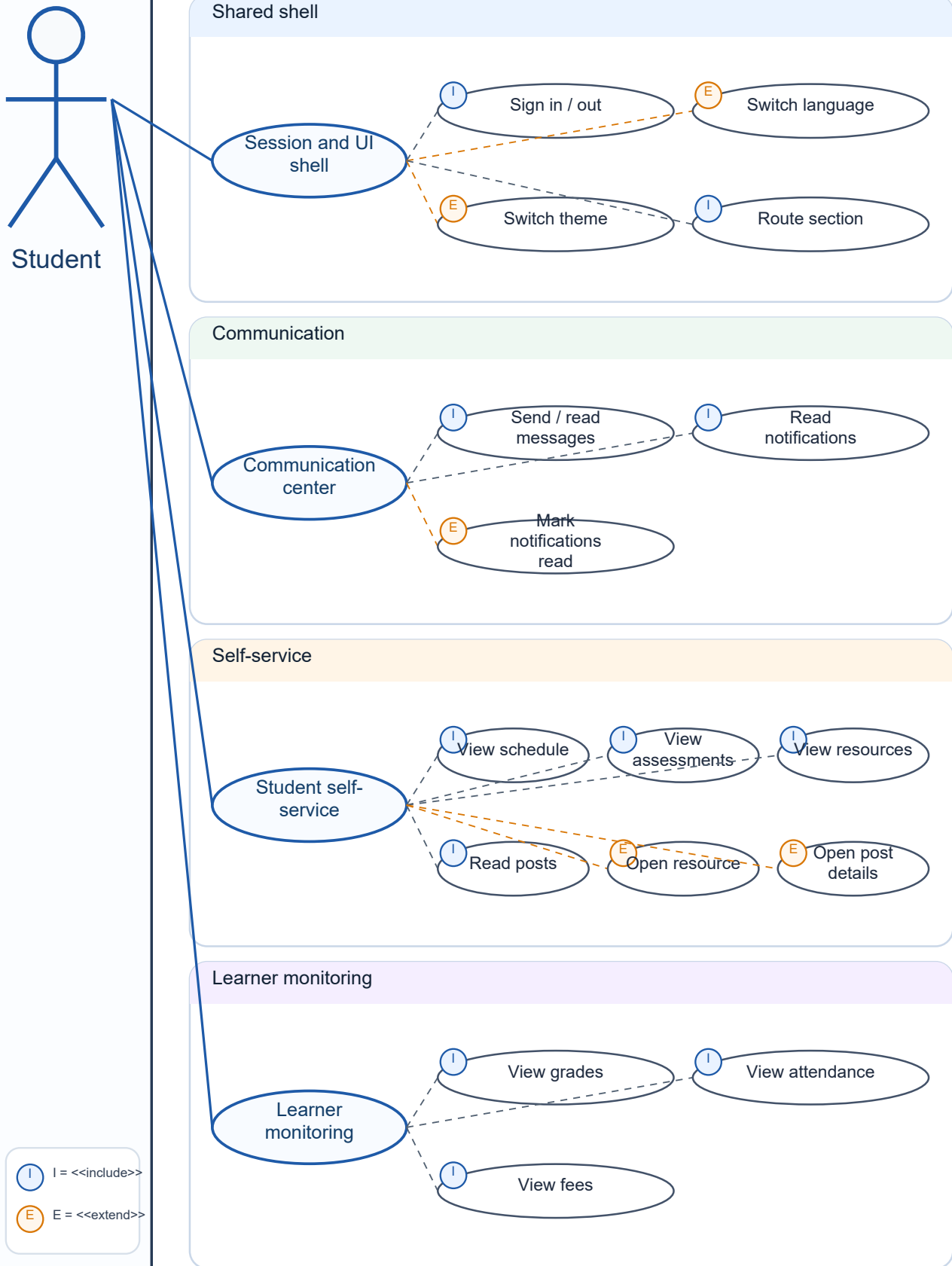
Use Case Diagram - Teacher

School Management System boundary



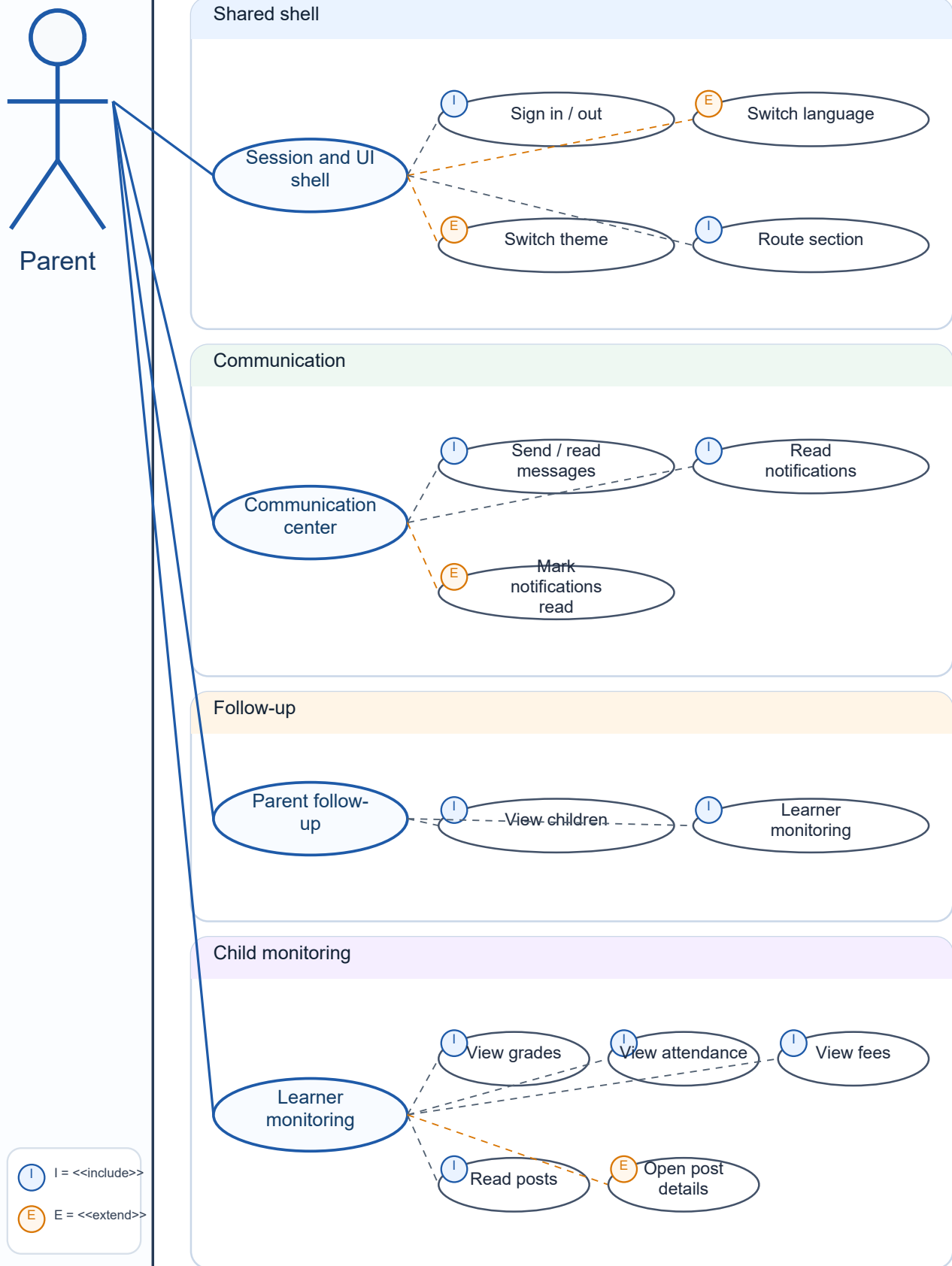
Use Case Diagram - Student

School Management System boundary



Use Case Diagram - Parent

School Management System boundary



Use Case Scope

Admin: complete system administration across people, academic, finance, posts, and notifications.

Receptionist: front-desk workflows for students, parents, records, finance lookup, posts, and communication.

Accountant: fees, payments, transactions, student financial files, notices, and attendance reports.

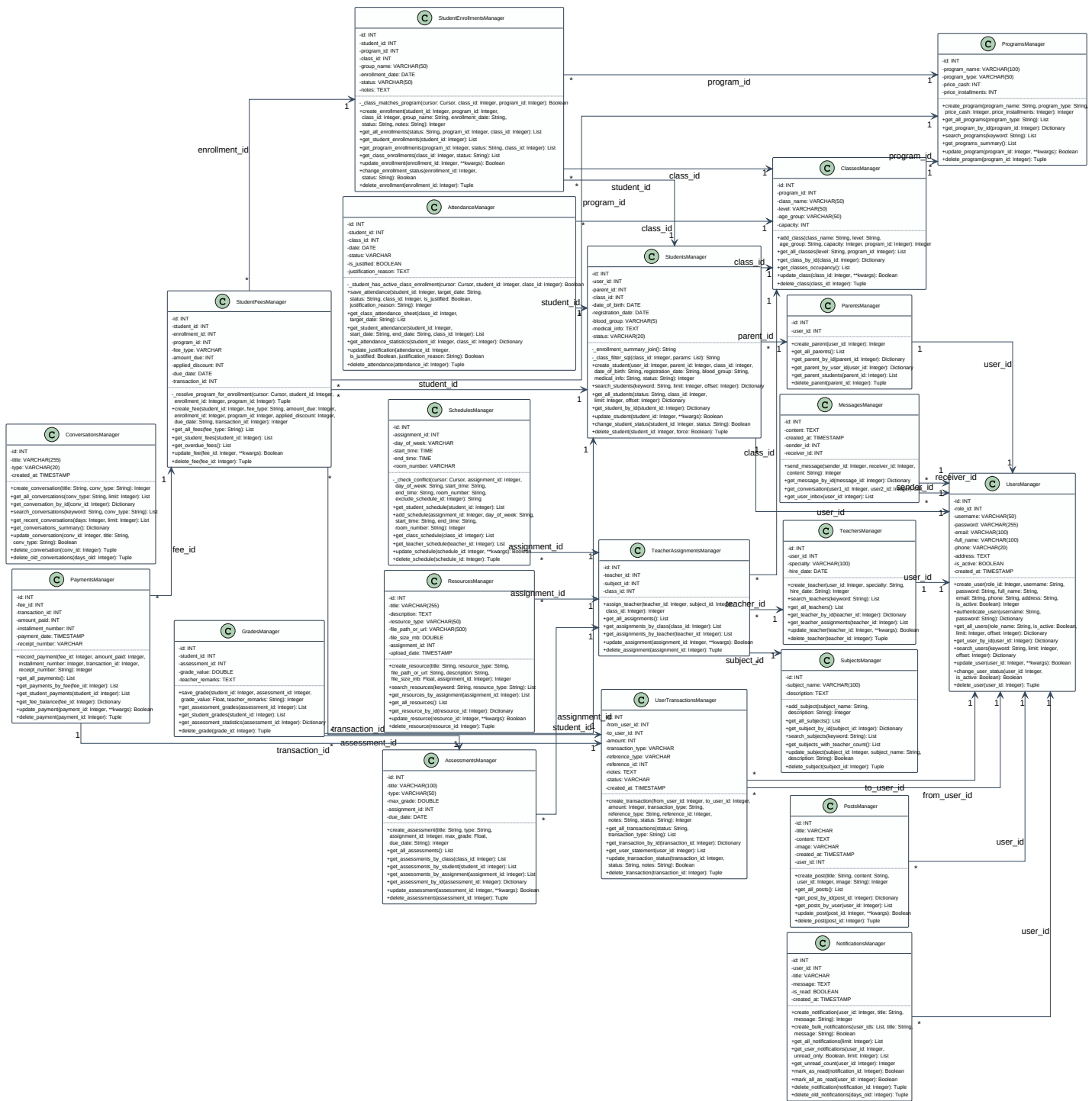
Teacher: assignments, schedules, attendance, assessments, grades, resources, messages, and notifications.

Student: read-only academic self-service: schedule, assessments, grades, attendance, resources, posts, fees, messages, and notifications.

Parent: child monitoring: grades, attendance, fees, posts, messages, and notifications.

Structural Model: Class Diagram

The class diagram presents the backend structure of the School Management System. It groups the database managers and domain entities used to support identity, academic management, learning operations, finance, communication, and publication features.



Structural Responsibilities

Identity and people: users, parents, students, teachers, authentication, and role ownership.

Academic structure: programs, classes, enrollments, subjects, and teacher assignments.

Learning operations: schedules, attendance, assessments, grades, and educational resources.

Finance: student fees, payments, and user transactions.

Communication: conversations, messages, notifications, and posts.

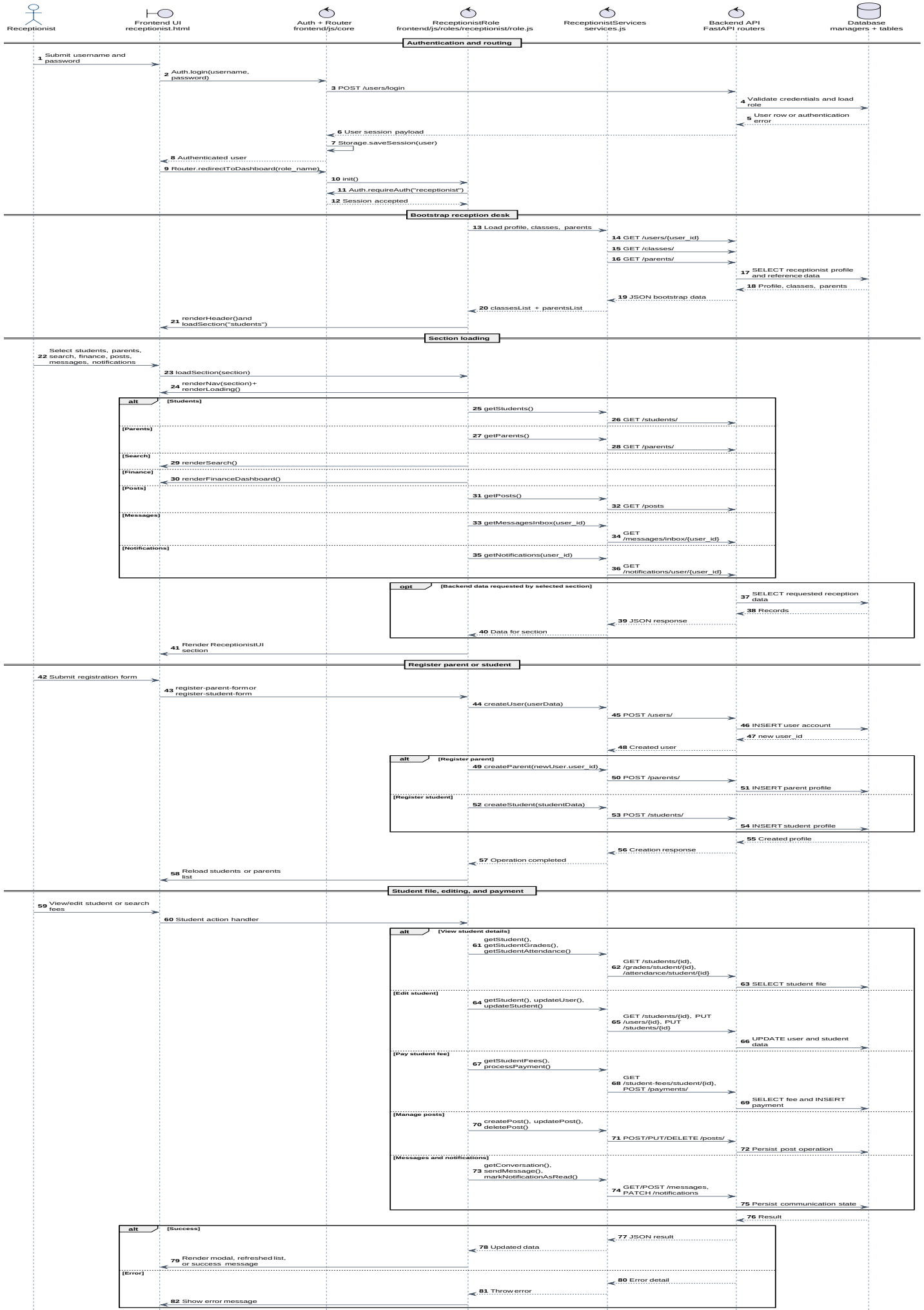
Dynamic Models: Sequence Diagrams

The sequence diagrams describe how each role interacts with the application at runtime. The common pattern is: user action in the frontend, role controller handling, service/API request, FastAPI router processing, database operation, then UI rendering of the result.

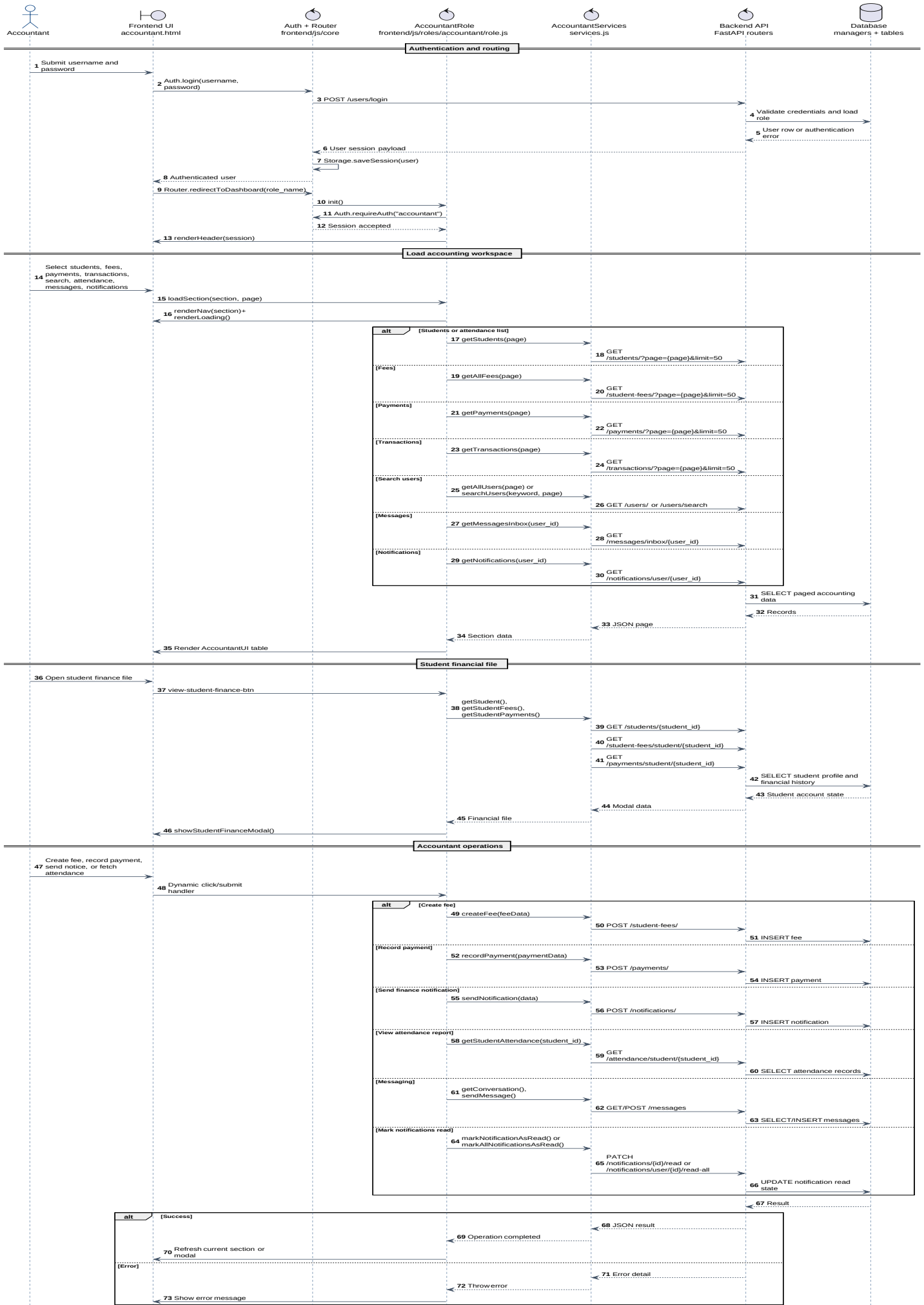
Sequence Diagram - Admin



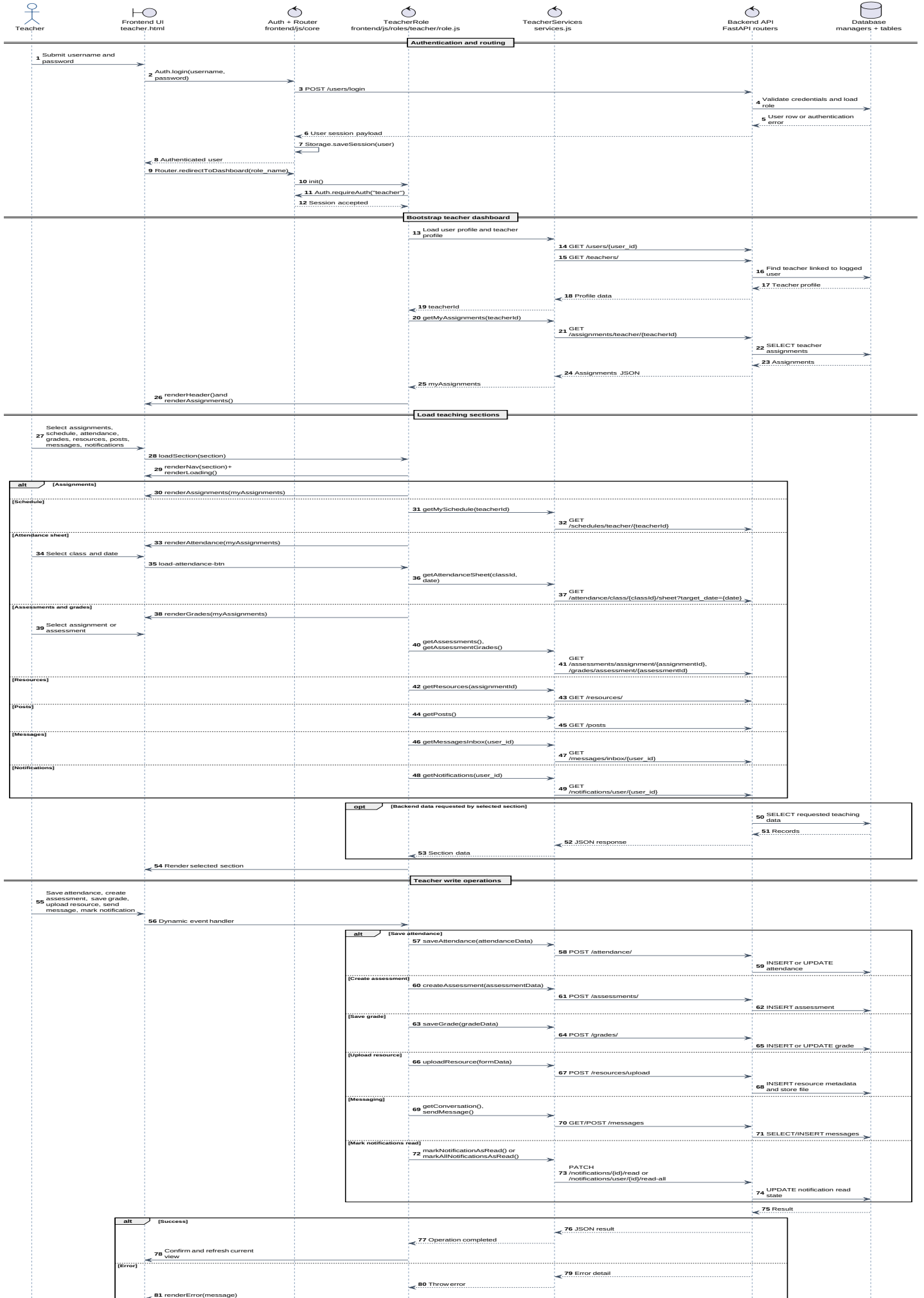
Sequence Diagram - Receptionist



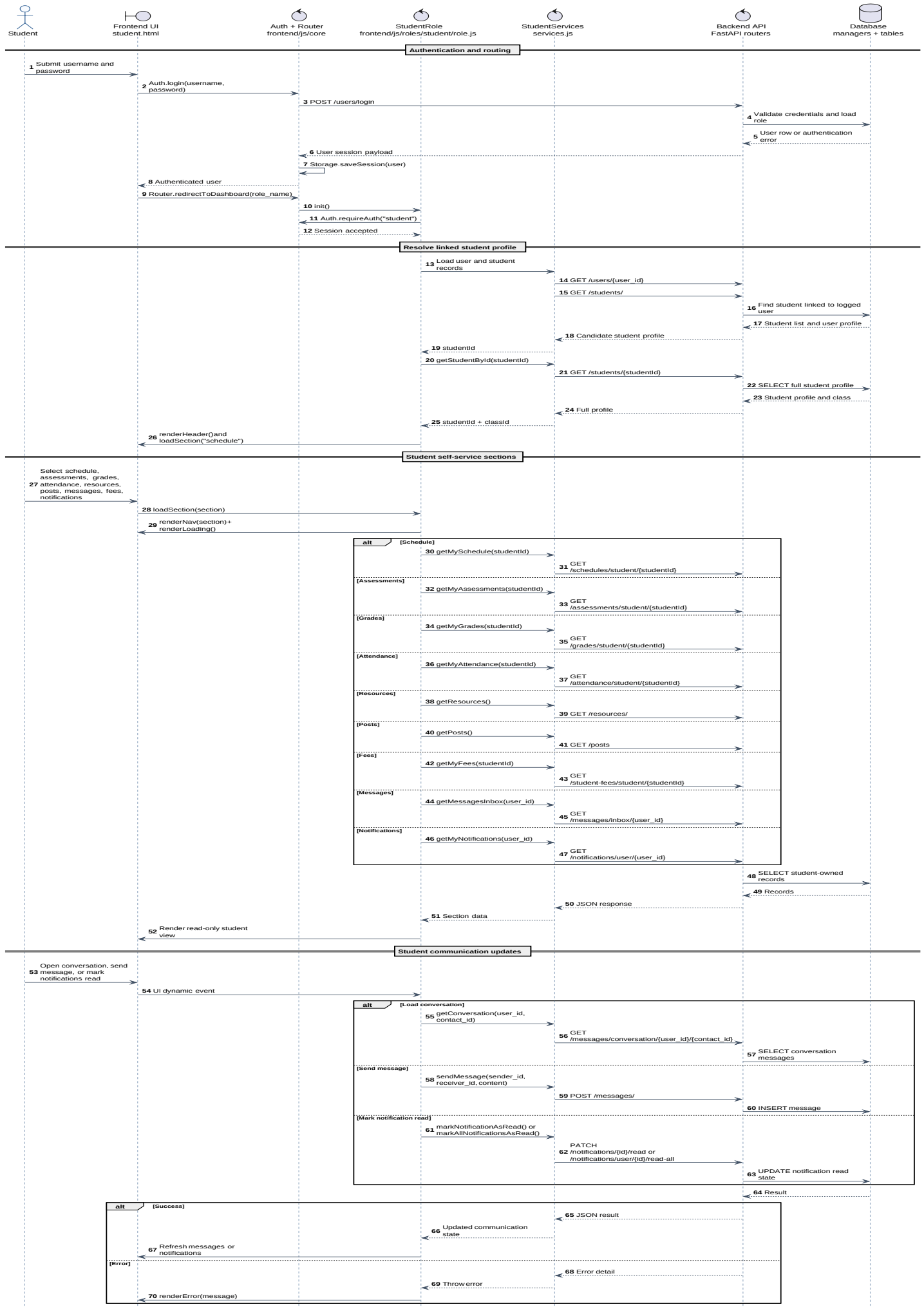
Sequence Diagram - Accountant



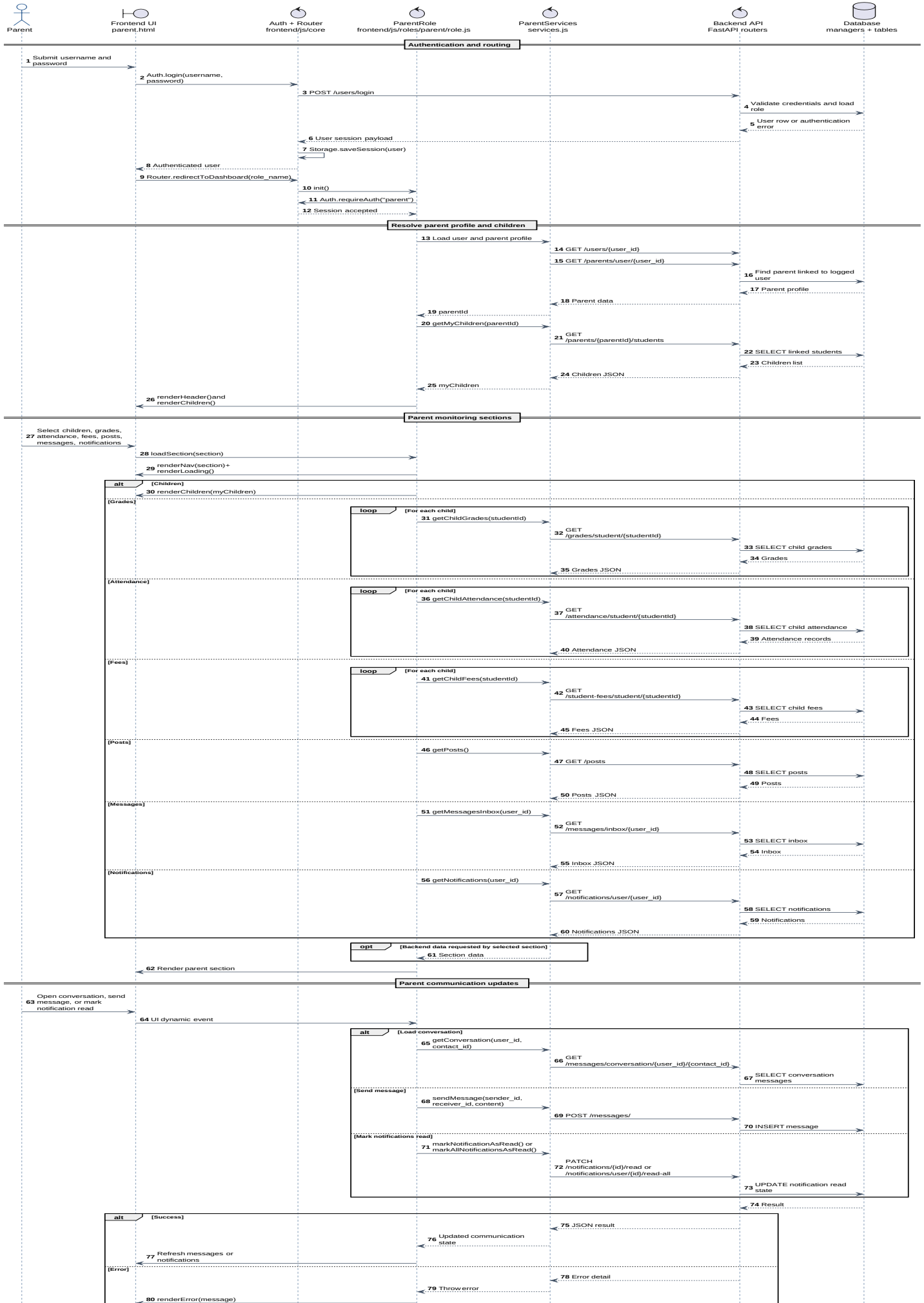
Sequence Diagram - Teacher



Sequence Diagram - Student



Sequence Diagram - Parent



Sequence Flow Summary

Admin: authentication, route selection, data loading, and full create/update/delete administration.

Receptionist: dashboard bootstrap, student and parent registration, student records, payments, posts, messaging, and notifications.

Accountant: finance workspace loading, student financial files, fee creation, payment recording, notices, attendance reports, messaging, and notification updates.

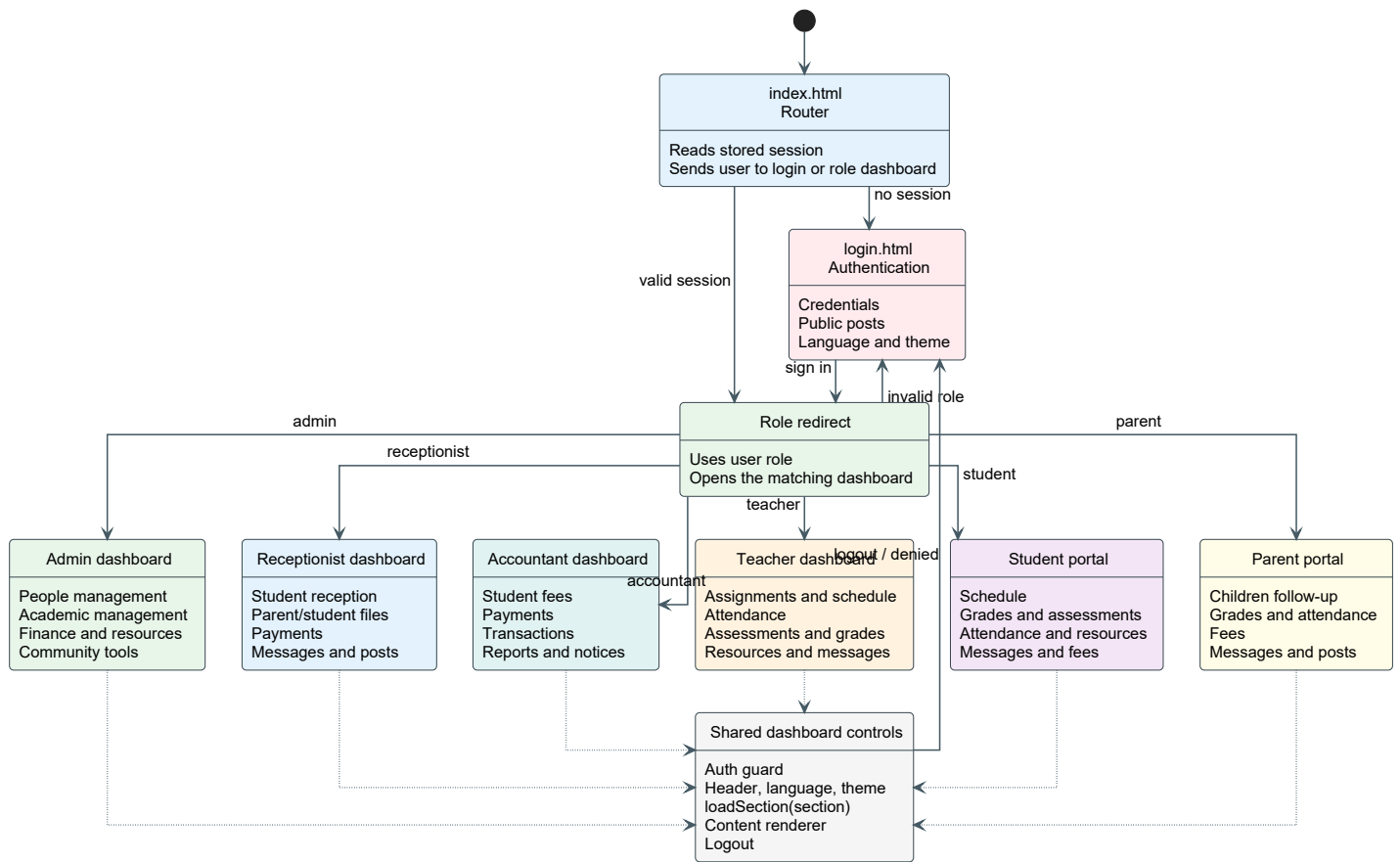
Teacher: teacher profile resolution, assignments, schedules, attendance, assessments, grades, resources, messaging, and notifications.

Student: linked profile resolution, academic self-service views, messages, and notifications.

Parent: parent profile resolution, linked children aggregation, child monitoring, posts, messages, and notifications.

User Interface: Navigation Diagram

The navigation diagram documents how users move from the entry point and authentication page into their role-specific dashboards. It also shows the shared frontend controls and the major section families exposed by each role.



Navigation Flow

Entry: users start from the index/login flow, where authentication determines the target dashboard.

Dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent each have a dedicated page and role controller.

Sections: navigation buttons call the active role controller, which loads the selected section and renders the corresponding UI.

Shared controls: authentication checks, language switching, theme preferences, logout, messaging, and notifications are reused across dashboards.

Technology Stack and Project Repository

This section lists the technologies directly used by the School Management System and its UML report.

Frontend Layer

The frontend is a static web application built with **HTML5**, **CSS3**, and **vanilla JavaScript**. It uses shared core modules for routing, authentication, local storage, API calls, preferences, and internationalization. Each role has its own page, controller, service layer, and UI renderer.

Backend API Layer

The backend is built with **Python** and **FastAPI**. The API is organized into routers for users, parents, students, teachers, classes, enrollments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, notifications, messages, posts, and programs. **Uvicorn** is used as the ASGI server and **Pydantic** supports request/response validation.

Database and Persistence

The persistence layer uses a relational database accessed through **mysql-connector-python** and **SQLAlchemy** utilities. Database managers separate identity, academic, learning, finance, and communication responsibilities.

Documentation and UML Report

UML diagrams are exported as **SVG** assets and embedded into the PDF report with **ReportLab** and **svglib**. The report covers use case diagrams, the class diagram, sequence diagrams, and the navigation diagram.

Artificial Intelligence Assistance

AI-assisted tools were used during the project to support analysis, code review, UML/report refinement, and documentation improvement. The final implementation and modeling decisions remain part of the team project work.

Project Repository

The project source code is available on GitHub:

<https://github.com/MohammedBoure/SMS-FE-BE>

Conclusion

Project Summary

The UML documentation provides a complete view of the School Management System. The use case diagrams define role responsibilities, the class diagram explains backend structure, the sequence diagrams show runtime frontend-to-backend collaboration, and the navigation diagram clarifies how users move through the interface.

Design Value

Together, the diagrams make the system easier to understand, review, and extend. They separate user responsibilities clearly, expose the core backend domains, and document the flow of data between the browser, API layer, and database.

Repository and Tooling Note

The project repository is available at <https://github.com/MohammedBoure/SMS-FE-BE>. AI-assisted tools were used to support development analysis, documentation refinement, and UML/report quality improvements.